

Sprawozdanie z Listy 1 (Laboratorium)

Obliczenia Naukowe

Agnieszka Głuszkiewicz

1 Zadanie 1: Rozpoznanie arytmetyki (macheps, eta, MAX)

1.1 Opis problemu

Zadanie polegało na eksperymentalnym wyznaczeniu stałych arytmetycznych: macheps (względna precyzja), **eta** (MIN_{sub} , najmniejsza liczba nieznormalizowana) oraz MAX (największa liczba skończona) dla typów Float16, Float32 i Float64.

1.2 Rozwiązanie

Stałe wyznaczyłam eksperymentalnie za pomocą iteracyjnych algorytmów: macheps poprzez poszukiwanie najmniejszej liczby ϵ takiej, że $1 + \epsilon > 1$; **eta** poprzez iteracyjne dzielenie liczby jeden, aż wynik zbiegnie do najmniejszej dodatniej wartości różnej od zera dla danego typu zmiennopozycyjnego; MAX poprzez mnożenie dwójki, aż nastąpi Overflow i później uzupełnienie bitów mantysy. Wyniki porównałam z wbudowanymi funkcjami `eps()`, `nextfloat()` oraz `floatmax()`.

1.3 Wyniki i interpretacja

Wyniki uzyskane iteracyjnie są zgodne z wartościami zwróconymi przez wbudowane funkcje.

Table 1: Porównanie stałych arytmetycznych IEEE 754: Julia vs. C/IEEE

Typ	Parametr	Wartość Iteracyjna	Wartość Funkcji	Zgodność
Float16	macheps	0.000977	0.000977	Tak
	η (MIN_{sub})	6.0e-8	6.0e-8	Tak
	MIN_{nor} (floatmin)	6.104e-5	6.104e-5	Tak
	MAX (floatmax)	6.55e4	6.55e4	Tak
Float32	macheps	1.1920929e-7	1.1920929e-7	Tak
	η (MIN_{sub})	1.0e-45	1.0e-45	Tak
	MIN_{nor} (floatmin)	1.1754944e-38	1.1754944e-38	Tak
	MAX (floatmax)	3.4028235e38	3.4028235e38	Tak
Float64	macheps	2.220446049250313e-16	2.220446049250313e-16	Tak
	η (MIN_{sub})	5.0e-324	5.0e-324	Tak
	MIN_{nor} (floatmin)	2.2250738585072014e-308	2.2250738585072014e-308	Tak
	MAX (floatmax)	1.7976931348623157e308	1.7976931348623157e308	Tak

Wartości Stałych z pliku `float.h` (C):

- Float32:
 - FLT_EPSILON (macheps): $1.19209290e - 07$
 - FLT_MIN (MIN_{nor}): $1.17549435e - 38$
 - FLT_MAX (MAX): $3.40282347e + 38$

- Float64:
 - DBL_EPSILON (macheps): $2.2204460492503131e - 16$
 - DBL_MIN (MIN_{nor}): $2.2250738585072014e - 308$
 - DBL_MAX (MAX): $1.7976931348623157e + 308$

Wszystkie stałe uzyskane z `float.h` (C) z dokładnością do zaokrąglenia są równe wynikom z tabeli wyżej.

1.4 Odpowiedzi na pytania

- **Związek macheps z precyzją (ϵ):** Epsilon jest często utożsamiany z macheps lub definiowany jako $\epsilon = \frac{1}{2}\text{macheps}$.
- **Związek eta z MIN_{sub} :** Liczba **eta** jest równa MIN_{sub} , czyli najmniejszej dodatniej liczbie nieznormalizowanej.
- **Co zwraca `floatmin(T)` i związek z MIN_{nor} :** Funkcje `floatmin(T)` zwracają MIN_{nor} , czyli najmniejszą dodatnią liczbę znormalizowaną w danym typie.

1.5 Wnioski

Eksperyment potwierdza, że arytmetyka zmiennoprzecinkowa w Julii jest zgodna ze standardem IEEE 754, co zostało zweryfikowane przez równe wartości stałych uzyskane iteracyjnie, za pomocą wbudowanych funkcji oraz poprzez porównanie ze stałymi z pliku nagłówkowego `float.h` języka C.

2 Zadanie 2: Formuła Kahana dla macheps

2.1 Opis problemu

Sprawdzenie, czy wzór Kahana: $3(4/3 - 1) - 1$ poprawnie wyznacza macheps.

2.2 Rozwiązanie

Obliczyłam wartość wzoru Kahana w precyzjach Float16, Float32 i Float64 i porównałam ją z wartościami macheps zwróconymi przez funkcję `eps()` dla danego typu.

2.3 Wyniki i interpretacja

Table 2: Wyniki formuły Kahana w różnych precyzjach

Typ	Wynik Kahana	macheps (<code>eps(T)</code>)	Zgodność
Float16	-0.000977	0.000977	Tak (wartość bezwzględna)
Float32	1.1920929e-7	1.1920929e-7	Tak
Float64	-2.220446049250313e-16	2.220446049250313e-16	Tak (wartość bezwzględna)

2.4 Wnioski

Formuła Kahana poprawnie wyznacza wartość macheps (z dokładnością do wartości bezwzględnej) we wszystkich testowanych precyzjach.

3 Zadanie 3: Rozmieszczenie liczb maszynowych

3.1 Opis problemu

Weryfikacja rozmieszczenia liczb Float64 w przedziałach $[1, 2]$, $[\frac{1}{2}, 1]$ i $[2, 4]$ oraz określenie kroku δ .

3.2 Rozwiązanie

Obliczyłam różnicę δ między $\text{nextfloat}(x)$ i x dla kilku x znajdujących się w okolicach granic danego przedziału, np. 1 i 2. Wykorzystałam zależność $\delta = \text{macheps} \cdot 2^E$. Dodatkowo sprawdziłam reprezentację bitową sprawdzanych liczb maszynowych za pomocą `bitstring()` dla przedziałów z zadania.

- **Przedział $[1, 2]$:** $E = 0$. Krok $\delta = 2^0 \cdot 2^{-52} = 2^{-52}$. Reprezentacja: $\mathbf{x} = 1 + \mathbf{k} \cdot 2^{-52}$, gdzie $k = 1, 2, \dots, 2^{52} - 1$.
- **Przedział $[\frac{1}{2}, 1]$:** $E = -1$. Krok $\delta = 2^{-1} \cdot 2^{-52} = 2^{-53}$. Reprezentacja: $\mathbf{x} = \frac{1}{2} + \mathbf{k} \cdot 2^{-53}$, gdzie $k = 1, 2, \dots, 2^{52}$.
- **Przedział $[2, 4]$:** $E = 1$. Krok $\delta = 2^1 \cdot 2^{-52} = 2^{-51}$. Reprezentacja: $\mathbf{x} = 2 + \mathbf{k} \cdot 2^{-51}$, gdzie $k = 1, 2, \dots, 2^{52} - 1$.

3.3 Wyniki i interpretacja

Krok δ dla Float64 (Testy nextfloat)

Eksperymentalnie sprawdziłam wartości $\text{nextfloat}(x) - x$ dla granic przedziałów:

- Dla $\mathbf{x} = \mathbf{1.0}$: $\delta = \text{nextfloat}(1.0) - 1.0 = 2^{-52}$.
- Dla $\mathbf{x} = \mathbf{0.5}$: $\delta = \text{nextfloat}(0.5) - 0.5 = 2^{-53}$.
- Dla $\mathbf{x} = \mathbf{2.0}$: $\delta = \text{nextfloat}(2.0) - 2.0 = 2^{-51}$.

Analiza Bitstring

Poniższe tabele przedstawiają reprezentację bitową pierwszych i ostatnich liczb maszynowych w trzech przedziałach: **[0.5, 1.0]**, **[1.0, 2.0]** oraz **[2.0, 4.0]**. Ilustrują one stałość kroku δ w reprezentacji **Float64 (IEEE 754)**.

Table 3: Reprezentacja binarna liczb maszynowych w przedziale $[0.5, 1.0]$ w Float64, $\delta = 2^{-53}$ [illegible]

4 Zadanie 4: $x \cdot (1/x)$ różne od 1

4.1 Opis problemu

Poszukiwanie w arytmetyce Float64 takiej wartości $\mathbf{x}$, dla której zachodzi $\mathbf{x} \cdot (1/\mathbf{x}) = 1$.

4.2 Rozwiązanie

Sprawdziłam kolejne liczby maszynowe $\mathbf{x}$ w przedziale $(1, 2)$ i obliczyłam $\mathbf{r} = \mathbf{x} \cdot (1/\mathbf{x})$, szukając wyniku $\mathbf{r} \neq 1$.

4.3 Wyniki i interpretacja

Oto wyniki, które pokazują, że $\mathbf{x} \cdot (1/\mathbf{x})$ nie zawsze daje 1:

- Przykładem jest $x = 1.5000000000000002$, dla którego $\mathbf{x} \cdot (1/\mathbf{x}) = 0.9999999999999999$.
- Najmniejszą znaną wartością $\mathbf{x}$ w przedziale $(1, 2)$ dającą wynik $\neq 1$ jest $\mathbf{x}_{\min} = 1.000000057228997$.

4.4 Wnioski

Eksperyment pokazuje, że arytmetyka zmiennoprzecinkowa, ze względu na swoją skończoną precyzję reprezentacji liczb, nie zawsze zwraca dokładne wyniki, nawet w przypadku prostych operacji arytmetycznych.

5 Zadanie 5: Obliczanie iloczynu skalarnego

5.1 Opis problemu

Porównanie dokładności iloczynu skalarnego $\sum x_i y_i$ dla czterech algorytmów sumowania w precyzjach Float32 i Float64.

5.2 Rozwiązanie

Obliczyłam iloczyn skalarny wektorów $\mathbf{x}$ i $\mathbf{y}$ przy użyciu czterech metod: sumowania w przód, w tył oraz sumowania po uprzednim posortowaniu składników od największego do najmniejszego oraz od najmniejszego do największego.

5.3 Wyniki i interpretacja

Właściwa wartość sumy: $1.00657107000000 \times 10^{-11}$.

Table 6: Wyniki iloczynu skalarnego w różnych precyzjach i algorytmach z błędami

Typ	Algorytm	Wynik Numeryczny	Błąd Bezwzględny	Błąd Względny
Float32	W przód (Forward)	-0.4999443	0.4999443	4.97×10^{10}
	W tył (Backward)	-0.4543457	0.4543457	4.51×10^{10}
	Max $\rightarrow$ Min	-0.5	0.4999999999	4.97×10^{10}
	Min $\rightarrow$ Max	-0.5	0.4999999999	4.97×10^{10}
Float64	W przód (Forward)	$1.0251881368296672 \times 10^{-10}$	$1.12584524 \times 10^{-10}$	11.18
	W tył (Backward)	$-1.5643308870494366 \times 10^{-10}$	$1.46367378 \times 10^{-10}$	14.54
	Max $\rightarrow$ Min	0.0	$1.00657107 \times 10^{-11}$	1.0
	Min $\rightarrow$ Max	0.0	$1.00657107 \times 10^{-11}$	1.0

Jak widać, błędy względne i bezwzględne są znacznie mniejsze przy zwiększonej precyzji i - w przypadku typu Float64 - nieco mniejsze w przypadku algorytmów wykorzystujących sortowanie, jednak otrzymane wartości nadal nie są satysfakcjonujące.

5.4 Wnioski

Eksperyment dowodzi, że w arytmetyce zmiennoprzecinkowej zarówno precyzja, jak i kolejność wykonywanych obliczeń mają istotny wpływ na dokładność wyników ze względu na błędy zaokrągleń.

6 Zadanie 6: Niestabilność numeryczna funkcji $f(x)$ i $g(x)$

6.1 Opis problemu

Porównanie stabilności numerycznej dwóch algebraicznie równoważnych funkcji dla małych x : $f(x) = \sqrt{x^2 + 1} - 1$ oraz $g(x) = \frac{x^2}{\sqrt{x^2 + 1} + 1}$.

6.2 Rozwiązanie

Obliczyłam wartości $f(x)$ i $g(x)$ dla bardzo małych wartości w arytmetyce Float64.

6.3 Wyniki i interpretacja

Table 7: Wartości funkcji $f(x)$ i $g(x)$ oraz stosunek f/g dla kolejnych $x = 8^{-n}$

n	$f(8^{-n})$	$g(8^{-n})$	f/g
1	$7.782218537318641 \cdot 10^{-3}$	$7.782218537318706 \cdot 10^{-3}$	0.9999999999999917
2	$1.2206286282867573 \cdot 10^{-4}$	$1.2206286282875901 \cdot 10^{-4}$	0.9999999999993178
3	$1.9073468138230965 \cdot 10^{-6}$	$1.907346813826566 \cdot 10^{-6}$	0.999999999998181
4	$2.9802321943606103 \cdot 10^{-8}$	$2.9802321943606116 \cdot 10^{-8}$	0.999999999999996
5	$4.656612873077393 \cdot 10^{-10}$	$4.6566128719931904 \cdot 10^{-10}$	1.0000000002328306
6	$7.275957614183426 \cdot 10^{-12}$	$7.275957614156956 \cdot 10^{-12}$	1.00000000003638
7	$1.1368683772161603 \cdot 10^{-13}$	$1.1368683772160957 \cdot 10^{-13}$	1.000000000000568
8	$1.7763568394002505 \cdot 10^{-15}$	$1.7763568394002489 \cdot 10^{-15}$	1.000000000000009
9	0.0	$2.775575615628914 \cdot 10^{-17}$	0.0
10	0.0	$4.336808689942018 \cdot 10^{-19}$	0.0
11	0.0	$6.776263578034403 \cdot 10^{-21}$	0.0
12	0.0	$1.0587911840678754 \cdot 10^{-22}$	0.0
13	0.0	$1.6543612251060553 \cdot 10^{-24}$	0.0
14	0.0	$2.5849394142282115 \cdot 10^{-26}$	0.0
15	0.0	$4.0389678347315804 \cdot 10^{-28}$	0.0

Dla $x = 8^{-9}$, funkcja $f(x)$ zwraca 0.0 w operacji odejmowania. Funkcja $g(x)$ zwraca poprawną, niezerową wartość $\approx 2.78 \times 10^{-17}$.

6.4 Wnioski

Funkcja $f(x)$ jest **niestabilna numerycznie** z powodu odejmowania liczb bardzo bliskich sobie dla małych x w operacji $\sqrt{x^2 + 1} - 1$, co prowadzi do błędnego 0.0. Funkcja $g(x)$, algebraicznie równoważna, jest **numerycznie stabilna**, ponieważ unika tej operacji odejmowania.

7 Zadanie 7: Obliczanie pochodnej (granica precyzji)

7.1 Opis problemu

Analiza dokładności numerycznego przybliżenia pochodnej ($\tilde{f}'(\mathbf{x}_0)$) za pomocą wzoru

$$f'(x_0) \approx \tilde{f}'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h}$$

w zależności od kroku $h = 2^{-n}$. Badam to dla funkcji:

$$f(x) = \sin x + \cos 3x$$

której właściwa wartość pochodnej wynosi **0.11694228168853815**.

7.2 Rozwiązanie

Obliczyłam błąd bezwzględny $|f' - \tilde{f}'|$ dla $n \in [0, 54]$, porównując przybliżenia z właściwą wartością.

7.3 Wyniki i interpretacja

Analiza wyników na kolejnej stronie uświadamia, że błąd $|f' - \tilde{f}'|$ maleje i osiąga minimum dla $n = 28$, po czym zaczyna znów rosnąć i osiąga duże wartości.

7.4 Odpowiedzi na pytania

- **Dlaczego zmniejszanie h obniża precyzję?** Malejący krok h powoduje odejmowanie niemal równych liczb w liczniku, co sprawia, że błąd zaokrąglenia staje się duży i pogarsza dokładność.
- **Co dzieje się dla $1 + h$?** Gdy h jest na tyle małe, że nie można go zapisać w danym typie (tutaj: $n \geq 53$), liczba $1.0 + h$ jest zaokrąglana do 1.0, co powoduje całkowitą utratę precyzji.

7.5 Wnioski

Błąd numeryczny w przybliżeniu pochodnej początkowo maleje wraz ze zmniejszaniem h , osiągając minimum przy $n = 28$. Dalsze zmniejszanie h prowadzi do rosnącego błędu zaokrąglenia, a przy $n \geq 53$ metoda całkowicie traci precyzję, ponieważ $x_0 + h$ jest zaokrąglane z powrotem do x_0 .

Table 8: Analiza błędu pochodnej dla $h = 2^{-n}$ (Float64)

n	$h = 2^{-n}$	$\tilde{f}'(x_0)$	Błąd $ f' - \tilde{f}' $	$1.0 + h$
0	1.0	2.0179892252685967	1.9010469435800585	2.0
1	0.5	1.8704413979316472	1.753499116243109	1.5
2	0.25	1.1077870952342974	0.9908448135457593	1.25
3	0.125	0.6232412792975817	0.5062989976090435	1.125
4	0.0625	0.3704000662035192	0.253457784514981	1.0625
5	0.03125	0.24344307439754687	0.1265007927090087	1.03125
6	0.015625	0.18009756330732785	0.0631552816187897	1.015625
7	0.0078125	0.1484913953710958	0.03154911368255764	1.0078125
8	0.00390625	0.1327091142805159	0.015766832591977753	1.00390625
9	0.001953125	0.1248236929407085	0.007881411252170345	1.001953125
10	0.0009765625	0.12088247681106168	0.0039401951225235265	1.0009765625
11	0.00048828125	0.11891225046883847	0.001969968780300313	1.00048828125
12	0.000244140625	0.11792723373901026	0.0009849520504721099	1.000244140625
13	0.0001220703125	0.11743474961076572	0.0004924679222275685	1.0001220703125
14	6.103515625e-5	0.11718851362093119	0.0002462319323930373	1.00006103515625
15	3.0517578125e-5	0.11706539714577957	0.00012311545724141837	1.000030517578125
16	1.52587890625e-5	0.11700383928837255	6.155759983439424e-5	1.0000152587890625
17	7.62939453125e-6	0.11697306045971345	3.077877117529937e-5	1.0000076293945312
18	3.814697265625e-6	0.11695767106721178	1.5389378673624776e-5	1.0000038146972656
19	1.9073486328125e-6	0.11694997636368498	7.694675146829866e-6	1.0000019073486328
20	9.5367431640625e-7	0.11694612901192158	3.8473233834324105e-6	1.0000009536743164
21	4.76837158203125e-7	0.1169442052487284	1.9235601902423127e-6	1.0000004768371582
22	2.384185791015625e-7	0.11694324295967817	9.612711400208696e-7	1.000000238418579
23	1.1920928955078125e-7	0.11694276239722967	4.807086915192826e-7	1.0000001192092896
24	5.960464477539063e-8	0.11694252118468285	2.394961446938737e-7	1.0000000596046448
25	2.9802322387695312e-8	0.116942398250103	1.1656156484463054e-7	1.0000000298023224
26	1.4901161193847656e-8	0.11694233864545822	5.6956920069239914e-8	1.0000000149011612
27	7.450580596923828e-9	0.11694231629371643	3.460517827846843e-8	1.0000000074505806
28	3.725290298461914e-9	0.11694228649139404	4.802855890773117e-9	1.0000000037252903
29	1.862645149230957e-9	0.11694222688674927	5.480178888461751e-8	1.0000000018626451
30	9.313225746154785e-10	0.11694216728210449	1.1440643366000813e-7	1.0000000009313226
31	4.656612873077393e-10	0.11694216728210449	1.1440643366000813e-7	1.0000000004656613
32	2.3283064365386963e-10	0.11694192886352539	3.5282501276157063e-7	1.0000000002328306
33	1.1641532182693481e-10	0.11694145202636719	8.296621709646956e-7	1.0000000001164153
34	5.820766091346741e-11	0.11694145202636719	8.296621709646956e-7	1.0000000000582077
35	2.9103830456733704e-11	0.11693954467773438	2.7370108037771956e-6	1.0000000000291038
36	1.4551915228366852e-11	0.116943359375	1.0776864618478044e-6	1.000000000014552
37	7.275957614183426e-12	0.1169281005859375	1.4181102600652196e-5	1.000000000007276
38	3.637978807091713e-12	0.116943359375	1.0776864618478044e-6	1.000000000003638
39	1.8189894035458565e-12	0.11688232421875	5.9957469788152196e-5	1.000000000001819
40	9.094947017729282e-13	0.1168212890625	0.0001209926260381522	1.0000000000009095
41	4.547473508864641e-13	0.116943359375	1.0776864618478044e-6	1.0000000000004547
42	2.2737367544323206e-13	0.11669921875	0.0002430629385381522	1.0000000000002274
43	1.1368683772161603e-13	0.1162109375	0.0007313441885381522	1.0000000000001137
44	5.684341886080802e-14	0.1171875	0.0002452183114618478	1.0000000000000568
45	2.842170943040401e-14	0.11328125	0.003661031688538152	1.0000000000000284
46	1.4210854715202004e-14	0.109375	0.007567281688538152	1.0000000000000142
47	7.105427357601002e-15	0.109375	0.007567281688538152	1.000000000000007
48	3.552713678800501e-15	0.09375	0.023192281688538152	1.0000000000000036
49	1.7763568394002505e-15	0.125	0.008057718311461848	1.0000000000000018
50	8.881784197001252e-16	0.0	0.11694228168853815	1.0000000000000009
51	4.440892098500626e-16	0.0	0.11694228168853815	1.0000000000000004
52	2.220446049250313e-16	-0.5	0.6169422816885382	1.0000000000000002
53	1.1102230246251565e-16	0.0	0.11694228168853815	1.0
54	5.551115123125783e-17	0.0	0.11694228168853815	1.0